ProAssess

Full Project Documentation

Platform	ProAssess — Staff Proficiency Assessment Platform
Stack	FastAPI · PostgreSQL · Chroma · Next.js · GPT-4o
Version	1.0.0
Classification	Confidential — Internal Use Only

Staff
Take assessments · View feedback

Line Manager
Create · Deploy · Manage

HR Admin
Knowledge base · Stats · Audit

System Admin
Full platform access

Table of Contents

PROASSESS — FULL PROJECT DOCUMENTATION

Audience: *Developers familiar with web development concepts (REST APIs, databases, React) but not necessarily expert-level in every technology used here.*

---

1. Project Overview

ProAssess is a staff proficiency assessment platform. It allows organisations to create, deploy, and evaluate assessments for their employees — powered by an AI pipeline that automatically generates questions from internal documents or general domain knowledge.

Core Objectives

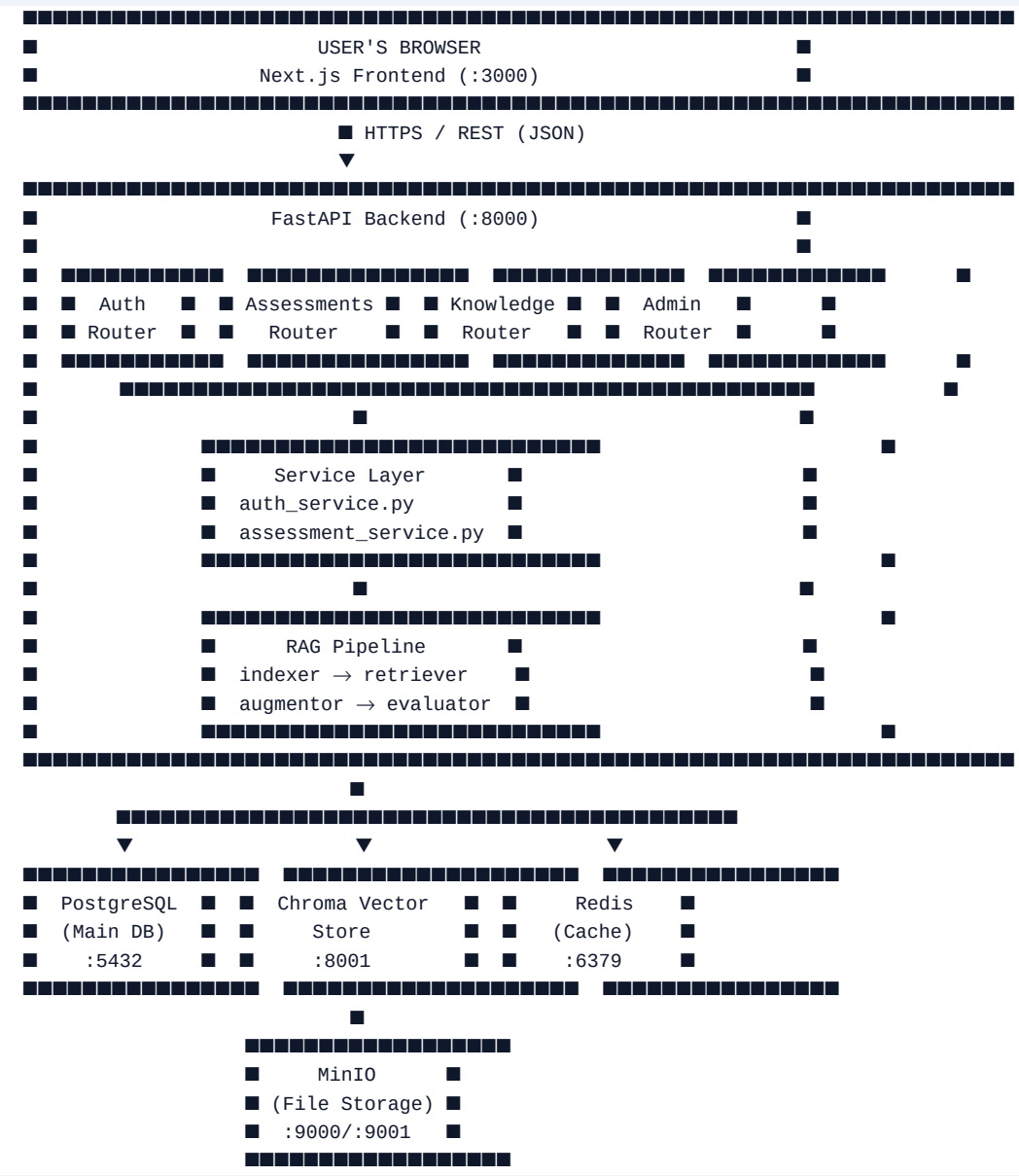
Objective	Description
Automated question generation	Line Managers specify a topic; the AI generates relevant questions without manual authoring
Multi-format assessments	Supports multiple choice (MCQ) and written/long-form questions
AI-powered grading	MCQ answers are graded instantly; written answers are evaluated by GPT with rubric-based scoring
Knowledge base integration	HR teams upload company documents (PDFs, Word files, spreadsheets) which the AI uses as source material
Role-based access	Different interfaces and permissions for Staff, Line Managers, HR Admins, and System Admins
Audit trail	Every significant action is logged for compliance and review

Who Uses It

- **Staff** — Take assessments assigned to them, receive instant feedback and scores
- **Line Managers (LM)** — Create assessments on topics relevant to their team, deploy them, and monitor status
- **HR Admins** — Manage the organisation's knowledge base, view org-wide statistics and audit logs
- **System Admins** — Full access across all roles

---

2. System Architecture



Request Flow — Creating an Assessment

```
LM submits form
      ■
      ▼
POST /api/v1/assessments/
      ■
      ▼
require_lm (JWT check + role guard)
      ■
      ▼
assessment_service.create_assessment()
  - Saves Assessment record (status: DRAFT)
  - Saves AssessmentTarget records
  - Kicks off background task
      ■
```

```
      ▼ (background, async)
RAG Pipeline
  ■■■ [1] Retriever  — finds relevant chunks from Chroma
  ■■■ [2] Augmentor  — sends context to GPT-4o, gets questions back
  ■■■ [3] Persists Question rows to PostgreSQL
```

Request Flow — Staff Taking an Assessment

```
Staff clicks "Start"
      ■
      ▼
POST /api/v1/assessments/{id}/start
      ■
      ▼
Creates StaffAssessment session
Returns questions (correct answers REMOVED)
      ■
Staff submits answers
      ■
      ▼
POST /api/v1/assessments/submit
      ■
      ■■■ MCQ answers → evaluate_mcq() [instant, deterministic]
      ■■■ Written answers → evaluate_written() [GPT-4o rubric scoring]
            ■
            ▼
      Scores stored, feedback returned immediately
```

---

3. Technology Stack

Backend

Technology	Version	Purpose
Python	3.12	Runtime
FastAPI	0.115	Web framework — async, automatic OpenAPI docs
SQLAlchemy	2.0	ORM (async) — maps Python classes to DB tables
Alembic	1.14	Database migration tool
asyncpg	0.30	Async PostgreSQL driver
Pydantic	2.x	Data validation and serialisation
LangChain	0.3	Orchestration for the LLM/RAG pipeline

Technology	Version	Purpose
OpenAI SDK	1.57	GPT-4o calls for question generation and grading
sentence-transformers	3.3	Cross-encoder model for result re-ranking
rank-bm25	0.2	Keyword-based search (BM25 algorithm)
python-jose	3.3	JWT creation and validation
passlib + bcrypt	1.7 / 4.0	Password hashing
uvicorn	0.32	ASGI server that runs FastAPI

Frontend

Technology	Version	Purpose
Next.js	16 (App Router)	React framework with server-side rendering support
TypeScript	5.x	Typed JavaScript
Tailwind CSS	3.x	Utility-first CSS framework
React	19	UI component library

Infrastructure

Service	Image	Purpose
PostgreSQL + pgvector	pgvector/pgvector:pg16	Primary relational database
Chroma	chromadb/chroma:0.5.20	Vector store for document embeddings
Redis	redis:7-alpine	Caching layer
MinIO	minio/minio:latest	S3-compatible object storage for uploaded files

---

4. Infrastructure & Services

PostgreSQL

The main database. Stores all structured data: users, organisations, assessments, questions, answers, audit logs. Uses the **`pgvector`** extension which adds vector similarity search capabilities directly in Postgres (not used heavily here since Chroma handles vectors, but available).

Chroma

A dedicated **vector database**. When a document is uploaded and indexed, it gets broken into small chunks (~512 tokens each), and each chunk is converted into a vector embedding (a list of ~3072 numbers that represents the meaning of that text). Chroma stores these embeddings and can quickly find the most semantically similar chunks to any query.

Think of it as a "meaning-aware search engine" — unlike a keyword search, it finds relevant content even when the exact words don't match.

Redis

An in-memory data store used for caching. Currently available in the stack for future use (e.g., caching frequently accessed assessments, rate limiting, session state).

MinIO

An open-source, self-hosted alternative to Amazon S3. When HR uploads documents (PDFs, DOCX, XLSX), they should be stored in MinIO. The current implementation has S3 upload marked as a TODO — files are processed in-memory.

---

5. Backend Deep Dive

Package Structure

The backend deliberately keeps most files at the **project root** rather than deep in nested packages. This was a design choice that keeps imports short, though it does mean the package **`__init__.py`** files act as re-export bridges.

```
/app (project root inside Docker)
■■■ main.py                  # App entry point — wires everything together
■■■ config.py                # All settings loaded from .env via Pydantic
■■■ database.py              # SQLAlchemy engine, session factory, Base class
■
■■■ auth.py                  # /auth/* routes
■■■ assessments.py           # /assessments/* routes
■■■ knowledge.py             # /knowledge/* routes
■■■ admin.py                 # /admin/* routes
■
■■■ auth_service.py          # JWT logic, password hashing, role guards
■■■ assessment_service.py    # All assessment business logic
■
■■■ indexer.py               # RAG stage 1: document loading &amp;amp; embedding
■■■ retriever.py             # RAG stage 2: hybrid search &amp;amp; re-ranking
```

```
■■■ augmentor.py            # RAG stage 3: question generation via GPT
■■■ evaluator.py            # RAG stage 4: MCQ scoring + GPT written eval
■
■■■ models/                 # SQLAlchemy ORM model definitions
■   ■■■ user.py
■   ■■■ assessment.py
■   ■■■ knowledge.py
■
■■■ schemas/                # Pydantic request/response models
■   ■■■ __init__.py
■
■■■ services/               # Package wrappers (re-export root service files)
■■■ rag/                    # Package wrappers (re-export root RAG files)
■■■ api/                    # Package wrapper (re-exports routers)
■
■■■ seed.py                 # Populates DB with test users and org
```

main.py — The Entry Point

`main.py` is where the FastAPI app is created and configured:

- **Lifespan context** — runs startup code (creates DB tables in dev, preloads the re-ranker model) and shutdown code
- **CORS middleware** — allows the frontend at **`localhost:3000`** and **`localhost:5173`** to make requests
- **Global exception handler** — catches any unhandled error and returns a clean 500 response instead of crashing
- **Router registration** — attaches the four API routers under **`/api/v1`**
- **Health check** — **`GET /health`** returns **`{"status": "ok"}`** for infrastructure monitoring

config.py — Settings

Uses **`pydantic-settings`** to read all configuration from environment variables (or **`.env`** file). Every setting has a sensible default for local development. Critical settings that must be overridden in production:

- **`SECRET_KEY`** — used to sign JWTs (must be a long random string)
- **`OPENAI_API_KEY`** — required for question generation and written answer evaluation

database.py — Async Database

Creates an **async** SQLAlchemy engine. "Async" here means database queries don't block the server — while waiting for Postgres to respond, FastAPI can handle other requests. This is important for a platform that may have many users taking assessments simultaneously.

The **`get_db`** function is a FastAPI dependency — routes declare they need a database session, and FastAPI injects one automatically, ensuring it's properly closed after each request.

---

6. The RAG Pipeline

RAG stands for **Retrieval-Augmented Generation**. The idea: instead of asking an AI to generate questions purely from its training data, you first retrieve relevant information from your own documents, then provide that as context to the AI. This produces more accurate, company-specific questions.

The pipeline has four stages:

Stage 1 — Indexer (`indexer.py`)

Runs when HR uploads a document or adds a URL.

```
Document (PDF/DOCX/XLSX/URL)
        ■
        ▼
Load &amp;amp; extract text
        ■
        ▼
Split into chunks (~512 tokens with 64-token overlap)
        ■
        ▼
Generate embedding for each chunk (OpenAI text-embedding-3-large)
        ■
        ▼
Store embeddings in Chroma + metadata in PostgreSQL
```

Chunking splits documents into overlapping segments so that context at chunk boundaries isn't lost. **Embedding** converts text into a 3072-dimension vector — two pieces of text about the same topic will have similar vectors even if they use different words.

Stage 2 — Retriever (`retriever.py`)

Runs when a new assessment is created to find relevant context.

```
Assessment topic + context prompt
        ■
        ▼
[Query Expansion] GPT rewrites topic into 4 diverse sub-queries
        ■
        ▼
[Dense Search] Chroma similarity search per sub-query (top-20 each)
        ■
[BM25 Search] Keyword search over the same candidate pool
        ■
        ▼
[Reciprocal Rank Fusion] Merges the two ranked lists into one
        ■
        ▼
[Cross-Encoder Re-ranking] Neural model scores each candidate
        ■
        ▼
Top-10 most relevant chunks returned
```

Why two search methods? Dense (vector) search is great for semantic similarity but can miss exact keyword matches. BM25 is great for keywords but misses paraphrasing. Combining them (hybrid search) is consistently

better than either alone.

Reciprocal Rank Fusion (RRF) is a simple but effective algorithm for merging two ranked lists. Each item scores `1/(rank + k)` in each list, and scores are summed. Items that rank highly in both lists score highest.

Cross-encoder re-ranking uses a separate neural model (`ms-marco-MiniLM`) that takes the query and each candidate together and scores their relevance. This is slower than embedding search but more accurate, so it's applied only to the top candidates as a final refinement step.

Stage 3 — Augmentor (`augmentor.py`)

Takes the retrieved context chunks and calls GPT-4o to generate questions.

```
Retrieved context chunks
        ■
        ▼
Build structured prompt with:
  - Assessment topic
  - Domain type (technical, compliance, etc.)
  - Retrieved context blocks
  - Requested question count
  - Question format (MCQ or written)
        ■
        ▼
GPT-4o generates questions as structured JSON
        ■
        ▼
JSON schema validation (retry up to 3x on malformed output)
        ■
        ▼
Questions stored in PostgreSQL with:
  - Question text
  - Options (for MCQ) + correct answer index
  - Model answer (for written)
  - Explanation
  - Source reference
  - Difficulty rating (1-5)
```

For **INDUSTRY** source assessments (no uploaded docs), GPT generates questions from its own training knowledge about the specified domain.

Stage 4 — Evaluator (`evaluator.py`)

Runs when staff submit their answers.

MCQ evaluation — fully deterministic, no AI needed:

```
is_correct = (given_index == correct_index)
score = 100.0 if is_correct else 0.0
```

Written evaluation — uses GPT-4o as a rubric-based marker:

```
Question + model answer + staff response
        ■
        ▼
GPT-4o prompt asking for:
  - Score (0-100)
  - Is the answer correct (boolean)
  - Specific feedback for the student
        ■
        ▼
Structured JSON response validated and stored
```

7. Frontend Deep Dive

App Router Structure

The frontend uses Next.js 15's App Router. Each folder under **`app/`** that contains a **`page.tsx`** file becomes a URL route.

```
app/
■■■ page.tsx                        → / (redirects to /login or /dashboard)
■■■ login/page.tsx                  → /login
■■■ dashboard/page.tsx              → /dashboard (redirects by role)
■
■■■ staff/
■   ■■■ assessments/
■       ■■■ page.tsx                → /staff/assessments (list)
■       ■■■ [id]/
■           ■■■ take/page.tsx       → /staff/assessments/:id/take
■           ■■■ feedback/page.tsx   → /staff/assessments/:id/feedback
■
■■■ lm/
■   ■■■ assessments/
■       ■■■ page.tsx                → /lm/assessments (list)
■       ■■■ new/page.tsx            → /lm/assessments/new (create form)
■       ■■■ [id]/page.tsx           → /lm/assessments/:id (manage)
■
■■■ hr/
    ■■■ page.tsx                    → /hr (stats + audit log)
    ■■■ knowledge/page.tsx          → /hr/knowledge (upload + index)
```

Authentication Flow

```
User submits login form
        ■
        ▼
POST /api/v1/auth/login
Returns { access_token, refresh_token }
        ■
        ▼
Tokens stored in localStorage
        ■
        ▼
```

```
GET /api/v1/auth/me
Returns user profile (id, email, name, role, org_id)
        ■
        ▼
User stored in React Context (AuthContext)
All pages read from this context
```

Token refresh is handled automatically in **`lib/api.ts`**. If any API request returns a 401 (token expired), the client silently uses the refresh token to get a new access token, then retries the original request. If the refresh also fails, the user is logged out and redirected to **`/login`**.

API Client (`lib/api.ts`)

All backend communication goes through a single **`apiFetch`** function that:

- Attaches the JWT **`Authorization`** header automatically
- Handles 401s with auto-refresh
- Throws typed errors for non-OK responses
- Returns typed TypeScript objects

Domain-specific functions (**`auth`**, **`assessments`**, **`knowledge`**, **`admin`**) are simple wrappers around **`apiFetch`** — routes and HTTP methods are defined once here, not scattered through components.

Role-Based Rendering

The **`Nav`** component and **`dashboard/page.tsx`** check **`user.role`** to show the right links and redirect to the right section:

Role value	Redirects to	Nav shows
`staff`	**`/staff/assessments`**	Assessments link
`lm`	**`/lm/assessments`**	Manage Assessments link
`hr_admin`	**`/hr`**	Stats + Knowledge Base links
`system_admin`	**`/lm/assessments`**	All links

---

8. Authentication & Roles

JWT Tokens

ProAssess uses **JSON Web Tokens (JWTs)**. A JWT is a signed string containing a payload — when a user logs in, the server creates a token containing their user ID, role, and organisation ID, then signs it with the **`SECRET_KEY`**. Any subsequent request that includes this token can be trusted without hitting the database, because the signature proves the token wasn't tampered with.

Two tokens are issued on login:

Token	Expiry	Purpose
Access token	8 hours	Sent with every API request in `Authorization: Bearer <token>` header
Refresh token	30 days	Only used to get a new access token when the old one expires

Role Guards

Routes are protected by FastAPI dependencies:

```
require_staff  # staff, lm, hr_admin, system_admin
require_lm     # lm, hr_admin, system_admin
require_hr     # hr_admin, system_admin
```

If a user calls an endpoint without the right role, they get a **`403 Forbidden`** response.

Password Hashing

Passwords are never stored in plain text. They are hashed with **bcrypt** — a deliberately slow algorithm designed to make brute-force attacks expensive. The salt is randomly generated per-password so identical passwords produce different hashes.

---

9. Data Models

Users & Organisations

```
Organisation (1) ■■■■ (many) User
Organisation (1) ■■■■ (many) Department
User (many) ■■■■■■■■■■ (many) Department  [via UserDepartment]
User (many) ■■■■■■■■■■ (many) SecurityGroup [via GroupMembership]
```

- Each **User** belongs to one **Organisation**
- Users can be in multiple **Departments** (with a job title and line manager per department)
- **SecurityGroups** are named groups (e.g. **`owner.engineering`**) used for targeting assessments

Assessments

```
Assessment (1) ■■■■ (many) Question
Assessment (1) ■■■■ (many) AssessmentTarget (dept or user IDs)
Assessment (1) ■■■■ (many) StaffAssessment (one per staff member)
StaffAssessment (1) ■■■ (many) StaffAnswer (one per question)
```

Assessment lifecycle:

```
DRAFT → DEPLOYED → CANCELLED
  ■
  ■■■ Questions generated in background (RAG pipeline)
       ■■■ Can only deploy once questions exist
```

StaffAssessment lifecycle:

```
NOT_STARTED → IN_PROGRESS → SUBMITTED → EVALUATED
```

Knowledge Base

```
KnowledgeSource (1) ■■■■ (many) DocumentChunk
```

- **KnowledgeSource** — a document or URL that's been indexed
- **DocumentChunk** — a single text segment (~512 tokens) from a source, with its Chroma vector ID for cross-reference

AuditLog

Every significant action writes an **`AuditLog`** record:

```
# Example entries
"CREATE_ASSESSMENT"
"DEPLOY_ASSESSMENT"
"CANCEL_ASSESSMENT"
"SUBMIT_ASSESSMENT"
```

Each entry stores: who did it, what they did, which resource, when, and optional JSON detail.

---

10. API Reference

Base URL: **`http://localhost:8000/api/v1`**

Auth

Method	Path	Role	Description
POST	/auth/login	Any	Email + password → access + refresh tokens
POST	/auth/refresh	Any	Refresh token → new token pair
GET	/auth/me	Any	Current user's profile

Assessments

Method	Path	Role	Description
POST	/assessments/	LM+	Create draft + trigger question generation
GET	/assessments/my	LM+	List assessments created by the caller
POST	/assessments/{id}/deploy	LM+	Move from DRAFT to DEPLOYED
POST	/assessments/{id}/cancel	LM+	Cancel a deployed assessment
DELETE	/assessments/{id}	LM+	Delete a DRAFT assessment only
GET	/assessments/available	Staff	List deployed assessments in caller's org
POST	/assessments/{id}/start	Staff	Begin a session, receive questions
POST	/assessments/submit	Staff	Submit answers, receive instant feedback
GET	/assessments/{id}/feedback	Staff	Retrieve past feedback for a session

Knowledge Base

Method	Path	Role	Description
GET	/knowledge/	LM+	List all knowledge sources
POST	/knowledge/upload	HR	Upload PDF/DOCX/XLSX for indexing

Method	Path	Role	Description
`POST`	`/knowledge/url`	HR	Index an external URL
`POST`	`/knowledge/{id}/reindex`	HR	Re-index an existing source
`DELETE`	`/knowledge/{id}`	HR	Soft-delete a source

Admin

Method	Path	Role	Description
`GET`	`/admin/stats`	HR	Org-wide statistics
`GET`	`/admin/audit-log`	HR	Paginated audit log

System

Method	Path	Role	Description
`GET`	`/health`	Any	Liveness check
`GET`	`/api/v1/pre-check`	Any	Client latency + browser compatibility check

Interactive docs: `http://localhost:8000/docs` *(development only)*

---

11. Environment Variables

Copy `.env.example` to `.env` and fill in required values before first run.

Variable	Required	Default	Description
`SECRET_KEY`	■	`change-me`	JWT signing key — use 64+ random characters in production
`OPENAI_API_KEY`	■	—	OpenAI API key for GPT-4o and embeddings
`APP_ENV`		`development`	`development` or `production`
`DATABASE_URL`		localhost default	Async PostgreSQL connection string

Variable	Required	Default	Description
`REDIS_URL`		localhost default	Redis connection string
`CHROMA_HOST`		`localhost`	Chroma vector store host
`CHROMA_PORT`		`8001`	Chroma port
`S3_ENDPOINT_URL`		MinIO default	Object storage endpoint
`S3_ACCESS_KEY`		`minioadmin`	Object storage access key
`S3_SECRET_KEY`		`minioadmin`	Object storage secret key
`OPENAI_CHAT_MODEL`		`gpt-4o`	Model for question generation and evaluation
`OPENAI_EMBEDDING_MODEL`		`text-embedding-3-large`	Model for document embeddings
`CORS_ORIGINS`		`["http://localhost:3000"]`	Allowed frontend origins
`ACCESS_TOKEN_EXPIRE_MINUTES`		`480` (8 hours)	How long access tokens last
`REFRESH_TOKEN_EXPIRE_DAYS`		`30`	How long refresh tokens last

12. Running the Project

Prerequisites

- Docker Desktop (running)
- Node.js 20+ (for the frontend)
- Python 3.11+ (only needed to run `seed.py` locally)

Start Everything

```
# 1. Copy environment config
cp .env.example .env
# Edit .env — set SECRET_KEY and OPENAI_API_KEY at minimum
# 2. Start all backend services (Postgres, Redis, Chroma, MinIO, FastAPI)
docker compose up -d
# 3. Seed the database with test users (first time only)
pip install bcrypt sqlalchemy asyncpg pydantic-settings
python seed.py
# 4. Start the frontend (in a separate terminal)
cd ../proassess-frontend
npm install
```

```
npm run dev
```

Test Users (after seeding)

All test users use the password **`Password123!`**

Email	Role	Access
`hr@acme.com`	HR Admin	Stats, audit log, knowledge base
`lm.eng@acme.com`	Line Manager	Create/manage assessments
`lm.sales@acme.com`	Line Manager	Create/manage assessments
`staff1@acme.com`	Staff	Take assessments, view feedback
`staff2–4@acme.com`	Staff	Take assessments, view feedback

Verify Services Are Running

```
curl http://localhost:8000/health          # → {"status":"ok","env":"development"}
curl http://localhost:3000                 # → Login page
```

Docker Dashboard or **`docker ps`** will show all five containers.

---

13. Database Migrations

ProAssess uses **Alembic** for database schema migrations. A migration is a versioned script that describes how to change the database schema — adding a column, creating a table, etc.

```
# After changing a model, generate a new migration automatically
alembic revision --autogenerate -m "describe_your_change"
# Apply all pending migrations to bring the DB up to date
alembic upgrade head
# Roll back the last migration
alembic downgrade -1
# See current migration status
alembic current
```

In **development**, **`main.py`** calls **`create_tables()`** on startup which uses SQLAlchemy to create any missing tables automatically. This is convenient but bypasses migration history — use Alembic properly in any shared or production environment.

---

14. Project File Structure

```
proassess-backend/              ← Backend (this repo)
├── main.py                     App entry point
├── config.py                   Settings (reads .env)
├── database.py                 Async SQLAlchemy setup
├── seed.py                     Dev database seeder
│
├── auth.py                     Auth API routes
├── assessments.py              Assessment API routes
├── knowledge.py                Knowledge base API routes
├── admin.py                    Admin/analytics API routes
│
├── auth_service.py             JWT, password hashing, role guards
├── assessment_service.py       Assessment business logic
│
├── indexer.py                  RAG: document loading + embedding
├── retriever.py                RAG: hybrid search + re-ranking
├── augmentor.py                RAG: GPT question generation
├── evaluator.py                RAG: answer scoring
│
├── models/
│   ├── __init__.py             Re-exports all models
│   ├── user.py                 User, Org, Dept, SecurityGroup models
│   ├── assessment.py           Assessment, Question, StaffAssessment models
│   ├── knowledge.py            KnowledgeSource, DocumentChunk, AuditLog models
│
├── schemas/
│   ├── __init__.py             All Pydantic request/response schemas
│
├── services/
│   ├── auth_service.py         (mirrors root auth_service.py)
│   ├── assessment_service.py   (mirrors root assessment_service.py)
│
├── rag/
│   ├── __init__.py             Pipeline orchestrator + re-exports
│   ├── indexer.py              (mirrors root indexer.py)
│   ├── retriever.py            (mirrors root retriever.py)
│   ├── augmentor.py            (mirrors root augmentor.py)
│   ├── evaluator.py            (mirrors root evaluator.py)
│
├── api/
│   ├── __init__.py             Re-exports all routers
│
├── alembic/                    Migration history
├── Dockerfile                  Container definition
├── docker-compose.yml          All services wired together
├── requirements.txt            Python dependencies
├── .env.example                Environment variable template
proassess-frontend/             ← Frontend (separate folder)
├── app/
│   ├── layout.tsx              Root layout (AuthProvider + Nav)
│   ├── page.tsx                Root redirect
│   ├── login/page.tsx          Login form
│   ├── dashboard/page.tsx      Role-based redirect
│   ├── staff/...               Staff pages
│   ├── lm/...                  Line Manager pages
│   ├── hr/...                  HR Admin pages
├── components/
│   ├── nav.tsx                 Top navigation bar
│   ├── spinner.tsx             Loading indicator
├── lib/
```

```
■   ■■■ api.ts                  API client + TypeScript types
■   ■■■ auth-context.tsx        React auth state + login/logout
■■■ .env.local                  Frontend environment config
```

15. Known Limitations & Future Work

Current Limitations

Area	Limitation
File storage	Document upload stores files in-memory only — S3/MinIO upload is scaffolded but not wired up
Assessment targeting	The target audience check (`_verify_user_is_target`) is a stub — any user in the org can take any deployed assessment
Tests	No test suite exists yet — `pytest tests/` is referenced in the README but the `tests/` directory needs to be created
Chroma persistence	Chroma is configured without a persistent client in `retriever.py` — embeddings may not survive container restarts without updating the Chroma connection to use a host
Written question retry	GPT evaluation of written answers has no retry logic — a failed API call would leave the answer unscored
Rate limiting	No API rate limiting is implemented

Natural Next Steps

- **Wire up S3 upload** — complete the `TODO` in `knowledge.py` to actually store files in MinIO
- **Add tests** — create `tests/` with pytest + httpx async test client
- **Implement targeting** — use `AssessmentTarget` and `UserDepartment` to properly control who sees which assessments
- **Add a results dashboard** — show LMs aggregated scores across their team
- **Email notifications** — notify staff when a new assessment is deployed to them
- **Production hardening** — disable `/docs`, enforce HTTPS, set up proper secrets management